

Lab03 - Questions complémentaires

Auteurs : Bleuer Rémy, Changanai Yoann, Rajadurai Thirusan

Classe : B

Groupe : 9

Date : 2 novembre 2025

Introduction

Ce laboratoire a pour objectif de développer une application Android permettant la saisie d'informations sur une personne (étudiant ou employé) via un formulaire. L'application doit gérer correctement les changements de configuration (rotation d'écran), valider les données saisies et offrir une expérience utilisateur fluide avec des contrôles adaptés (spinners, date picker, champs multi-lignes).

Implémentation

Choix d'architecture

Nous avons choisi d'utiliser le pattern **ViewModel** pour gérer la persistance des données lors des changements de configuration. Cette approche permet de conserver les données du formulaire même lors de la rotation de l'écran, sans avoir à gérer manuellement la sauvegarde/restauration via `onSaveInstanceState()`.

Justification : Le **ViewModel** est la solution recommandée par Google pour gérer l'état de l'UI de manière lifecycle-aware. Il est automatiquement conservé lors des changements de configuration et offre une séparation claire entre la logique métier et l'interface.

Modèle de données

Nous avons implémenté une hiérarchie de classes avec :

- ◆ Une classe abstraite **Person** contenant les attributs communs
- ◆ Deux classes concrètes **Student** et **Worker** héritant de **Person**

Justification : Cette architecture orientée objet permet de factoriser le code commun tout en permettant des spécialisations pour les étudiants et employés. Bien que Kotlin ne permette pas l'héritage avec les data classes, cette approche reste la plus maintenable.

Validation des Spinners

Pour les spinners (nationalité et secteur), nous avons ajouté un élément "Sélectionner" en position 0 et implémenté une validation via des flags booléens (`isNationalityValid`, `isSectorValid`).

Justification : Plutôt que d'afficher des messages d'erreur intrusifs, nous avons choisi de désactiver le bouton OK tant que les spinners obligatoires ne sont pas valides. Cette approche offre un feedback visuel immédiat et non intrusif à l'utilisateur.

Gestion du DatePicker

Nous avons utilisé `DatePickerDialog` du SDK Android et géré le lifecycle manuellement en appelant `dismiss()` dans `onDestroy()`.

Justification : Bien que `DialogFragment` soit plus robuste pour gérer automatiquement la rotation, la solution manuelle avec `dismiss()` est plus simple à implémenter pour ce cas d'usage et suffit à éviter l'exception `WindowLeaked`.

Tests et manipulations

Tests effectués

1. Rotation d'écran :

- ◆ Saisie de données dans le formulaire
- ◆ Rotation de l'écran
- ◆ **Résultat :** Les données sont conservées grâce au ViewModel

2. Validation des champs :

- ◆ Test avec champs vides
- ◆ Test avec email invalide
- ◆ Test avec spinners non sélectionnés

- ◆ **Résultat** : Le bouton OK est désactivé tant que les spinners ne sont pas valides, et les autres validations affichent des erreurs appropriées

3. DatePicker et rotation : ✓

- ◆ Ouverture du DatePicker
- ◆ Rotation de l'écran
- ◆ **Résultat** : Pas d'exception WindowLeaked grâce au dismiss() dans onDestroy()

4. Navigation clavier : ✓

- ◆ Test du bouton "Suivant" sur chaque champ
- ◆ **Résultat** : Navigation fluide entre les champs avec
`imeOptions="actionNext"`

5. Champ multilignes : ✓

- ◆ Saisie de texte long avec retours à la ligne
- ◆ **Résultat** : Le champ s'adapte correctement grâce à
`inputType="textMultiLine"`

Tests de régression

- ◆ Changement entre mode Étudiant/Employé : ✓ Les champs spécifiques s'affichent/masquent correctement
- ◆ Bouton Cancel : ✓ Réinitialise tous les champs et les flags de validation
- ◆ Persistance après validation : ✓ Les données validées sont conservées dans le ViewModel



Mode d'emploi

1. Saisie des informations de base :

- ◆ Remplir Nom, Prénom
- ◆ Cliquer sur le champ Date de naissance pour ouvrir le sélecteur de date
- ◆ Sélectionner une nationalité (obligatoire)
- ◆ Remplir l'adresse email (validation automatique)

2. Choix du type de personne :

- ◆ Sélectionner "Étudiant" ou "Employé"
- ◆ Remplir les champs spécifiques qui apparaissent

3. Validation :

- ◆ Le bouton OK devient actif uniquement quand les spinners obligatoires sont sélectionnés
- ◆ Cliquer sur OK pour créer l'objet Person (visible dans les logs Logcat)

4. Annulation :

- ◆ Cliquer sur Cancel pour vider tous les champs



4.1 Pour le champ remark, destiné à accueillir un texte pouvant être plus long qu'une seule ligne, quelle configuration particulière faut-il faire dans le fichier XML pour que son comportement soit correct ? Nous pensons notamment à la possibilité de faire des retours à la ligne, d'activer le correcteur orthographique et de permettre au champ de prendre la taille nécessaire.

En y ajoutant l'attribut `android:inputType` avec les propriétés `"textMultiLine|textCapSentences|textAutoCorrect"` ou encore l'attribut `android:minLines="3"` pour que le champ ait déjà une hauteur minimale.

```
<EditText
    android:id="@+id/commentField"
    android:layout_width="0dp"
    android:layout_height="wrap_content"
    android:inputType="textMultiLine|textCapSentences"
    android:minLines="3"
    android:gravity="top|start"
    android:imeOptions="actionDone"
    android:layout_marginTop="24dp"
    android:hint="@string/hint_comment"
    app:layout_constraintTop_toBottomOf="@id/emailField"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintEnd_toEndOf="parent" />
```

4.2 Pour afficher la date sélectionnée via le DatePicker nous pouvons utiliser un DateFormat permettant par exemple d'afficher 12 juin 1996 à partir d'une instance de Date. Le formatage des dates peut être relativement différent en fonction des langues, la traduction des mois par exemple, mais également des habitudes régionales différentes : la même date en anglais britannique serait 12th June 1996 et en anglais américain June 12, 1996. Comment peut-on gérer cela au mieux ?

Pour afficher la date sélectionnée via le `DatePicker`, on utilise un `DateFormat` (dans notre cas `SimpleDateFormat("dd/MM/yyyy", Locale.getDefault())` pour nos standards en suisse) afin de formater un objet `Date` ou `Calendar` selon les habitudes de la langue et région. On peut ainsi adapter facilement le format suivant la langue/région.

On peut également utiliser les classes de formatage de date du système (`java.text.DateFormat.getDateInstance( ... )`) qui s'adaptent automatiquement à la locale de l'utilisateur. Cela permet que la date soit affichée dans le style correct pour l'utilisateur (langue, position jour/mois/année).

4.3 Veuillez choisir une question en fonction de votre choix d'implémentation :

- a. Si vous avez utilisé le `DatePickerDialog1` du SDK. En cas de rotation de l'écran du smartphone lorsque le dialogue est ouvert, une exception `android.view.WindowLeaked` sera présente dans les logs, à quoi est-elle due et comment pouvons-nous la corriger ?
- b. Si vous avez utilisé le `MaterialDatePicker2` de la librairie Material. Est-il possible de limiter les dates sélectionnables dans le dialogue ? Ceci en particulier pour une date

de naissance

car il est peu probable d'avoir une personne née il y a plus de 110 ans ou à une date dans le futur. Comment pouvons-nous mettre cela en place ?

Ayant utilisé le `DatePickerDialog` du SDK, si l'écran se tourne pendant que le dialogue est ouvert, on peut rencontrer l'exception `android.view.WindowLeaked`. Cette exception survient parce que le dialogue est toujours attaché à l'`Activity` précédente qui est détruite/recréée lors de la rotation et le `Dialog` n'a pas été détaché ou recréé correctement.

Pour corriger cela, on peut utiliser :

- ♦ un `DialogFragment` plutôt qu'un simple `DatePickerDialog` (car `DialogFragment` gère automatiquement le cycle de vie lié à la rotation)
- ♦ ou bien dans `onDestroy()` ou `onPause()` ferme explicitement le dialogue (par exemple `dialog.dismiss()` si il est encore ouvert). Ainsi on évite le « leak » de fenêtre.

Nous avons décidé d'utiliser la fermeture manuelle car plus simple à implémenter. Il faut faire attention néanmoins de rien oublier tel que le `.dismiss()` au `onDestroy()` de l'activité et autre gestion.

4.4 Lors du remplissage des champs textuels, vous pouvez constater que le bouton « suivant » présent sur le clavier virtuel permet de sauter automatiquement au prochain champ à saisir, cf. Fig. 2. Est-ce possible de spécifier son propre ordre de remplissage du questionnaire ? Arrivé sur le dernier champ, est-il possible de faire en sorte que ce bouton soit lié au bouton de validation du questionnaire ?

Hint : Le champ remark, multilignes, peut provoquer des effets de bords en fonction du clavier virtuel utilisé sur votre smartphone.

Vous pouvez l'échanger avec le champ e-mail pour faciliter vos recherches concernant la réponse à cette question.

Oui, il est possible de spécifier un ordre personnalisé de remplissage entre les champs textuels. En utilisant l'attribut

`android:nextFocusDown` , `android:nextFocusForward` , etc., ou les attributs `android:imeOptions="actionNext"`

sur les `EditText` pour définir que le bouton « suivant » du clavier passe au champ suivant désiré.

Par exemple, sur le champ email on peut mettre

`android:imeOptions="actionNext"` et sur le champ commentaire

`android:imeOptions="actionDone"` , ce qui fait que sur le dernier champ le

bouton du clavier devient « Terminer » ou « Done »

et déclenche la validation du formulaire. On peut écouter l'événement

`setOnEditorActionListener` sur ce dernier champ pour déclencher

le bouton de validation (par exemple appeler la méthode `onValidate()`) quand

l'utilisateur appuie sur « Done ». On garantit un flux

utilisateur fluide de champ en champ et finalement vers la validation.

4.5 Pour les deux Spinners (nationalité et secteur d'activité), comment peut-on faire en sorte que le premier choix corresponde au choix null, affichant par exemple le label « Sélectionner » ? Comment peut-on gérer cette valeur pour ne pas qu'elle soit confondue avec une réponse ?

Pour les deux `Spinner` (nationalité et secteur d'activité), nous avons ajouté un élément par défaut dans la string-array en position 0, qui a pour texte "Sélectionner". Ensuite dans le code, lors d'un changement de sélection, on vérifie et met à jour le flag de validation pour le spinner en question.

Pour simplifier la gestion d'erreur sans pop-up dérangeant, nous avons choisi de désactiver le bouton de validation tant que tous les flags des spinners ne soient pas valides.

Seealso

L'IA a été utilisée partiellement pour des vérifications mineures telles que l'orthographe, la grammaire, l'entrée d'informations redondantes (`onCancel()`). Elle a été évitée au maximum dans la réalisation de ce laboratoire pour des

raisons pédagogiques. D'autres sources ont pu être ici et là notamment **geeksforgeeks** et leurs exemple de **datepickerdialog**.

Conclusion personnelle

Ce laboratoire nous a permis de comprendre l'importance de la gestion du cycle de vie des activités Android, notamment lors des changements de configuration. L'utilisation du pattern ViewModel s'est révélée être une solution élégante et robuste pour gérer l'état de l'application.

La validation des formulaires côté client améliore significativement l'expérience utilisateur en offrant un feedback immédiat. Le choix de désactiver le bouton de validation plutôt que d'afficher des messages d'erreur s'est avéré judicieux pour éviter de surcharger l'interface.

Nous avons également apprécié la flexibilité du système de layouts Android (ConstraintLayout) qui permet de créer des interfaces adaptatives. La gestion des différents types d'input (clavier, date picker, spinners) nous a permis de comprendre les bonnes pratiques en matière d'UX mobile.

Points d'amélioration possibles :

- ◆ Ajout d'animations lors du changement entre mode Étudiant/Employé
- ◆ Feedback visuel lors de validation du champ